PRN232 FINAL ASSIGNMENT

# System Architecture & Database Design

## EXE Group Management System

*Table 1. Project information*

| Item | Details |
|---|---|
| **Course** | PRN232 – Building Cross-Platform Back-End Application with .NET |
| **Project** | EXE Group Management System |
| **Class / group** | SE18xx – Group N |
| **Semester** | Fall 2026 (FA26) |
| **Architectural style** | Microservices architecture, documented with the C4 model (Level 1 & Level 2) |
| **Team** | See Team Members & Responsibilities below |
| **Instructor** | PhuongLHK |
| **Document version** | 2.2 – November 5, 2026 |

## Team Members & Responsibilities

*Table 2. Team members and responsibilities*

| No. | Full name | Student ID | Main responsibilities |
|---|---|---|---|
| 1 | Member 1 | SExxxxxx | Platform and API Gateway: solution skeleton, shared building blocks, Docker Compose, CI, YARP gateway (JWT, Swagger UI), README and demo script |
| 2 | Member 2 | SExxxxxx | Identity Service (sign-in, JWT, users, roles) and Join Request Service REST API |
| 3 | Member 3 | SExxxxxx | Group Service REST API: semesters, courses, classes, rosters, groups, lecturer dashboard |
| 4 | Member 4 | SExxxxxx | gRPC GroupMembership server and client, Transactional Outbox, Redis Streams consumer, background jobs |
| 5 | Member 5 | SExxxxxx | React Single-Page App for all roles, nginx |

*Highlighted cells are placeholders – replace them with the real class, names and student IDs before submission, then delete this note.*

## Revision History

*Table 3. Revision history*

| Version | Date | Changes |
|---|---|---|
| 1.0 | October 14, 2026 | Initial architecture and database design. |
| 2.0 | November 1, 2026 | Container diagram redrawn left to right with fewer words (lecturer feedback) and context diagram aligned; all diagrams redrawn for portrait pages with straight, orthogonal connectors; class rosters added; Redis Streams chosen as message broker; gRPC and event contracts, join-request state machine with idempotent processing, complete REST API, background jobs and deployment defined; conceptual ERD reworked; physical design split per database with a reference DDL tested on SQL Server. |
| 2.1 | November 4, 2026 | An ALREADY_ADDED reply from AddMemberIfEligible now confirms the join request as Approved (new transition T11), so a lost GroupMemberAdded event can no longer leave a request in Processing; AddMemberIfEligibleReply gains added_at_utc. |

| Version | Date | Changes |
|---|---|---|
| 2.2 | November 5, 2026 | Stack moved to .NET 10, EF Core 10 and React 19; every route versioned (/api/v1); one response envelope replaces ProblemDetails; list queries use search, sort, page, pageSize, fields and expand; refresh tokens in scope (RefreshTokens table, POST /api/v1/auth/refresh-token and /logout); three projects per service enforced by architecture tests; C4 component, deployment and dynamic views generated from docs/architecture/workspace.dsl (Appendix C). |

## Table of Contents

1. Context ..... 4
2. Problems ..... 4
3. Solutions ..... 4
3.1. Proposed Solution ..... 4
3.2. Design Rationale – Why Both gRPC and Events ..... 5
3.3. Scope Definition ..... 5
4. Main Actors ..... 6
4.1. Actor Descriptions ..... 6
4.2. Permission Matrix ..... 6
5. Main Features & Business Rules ..... 7
5.1. Functional Requirements ..... 7
5.2. Business Rules ..... 7
5.3. Join Request Lifecycle ..... 8
5.4. RESTful API Endpoints ..... 9
5.4.1. Conventions ..... 9
5.4.2. Identity Service ..... 10
5.4.3. Group Service ..... 10
5.4.4. Join Request Service ..... 11
6. System Architecture – C4 Model (Level 1 & Level 2) ..... 13
6.1. Level 1: System Context Diagram ..... 13
6.2. Level 2: Container Diagram ..... 14
6.3. Container Responsibilities ..... 14
6.4. Inter-Service Workflow – Approving a Join Request ..... 15
6.5. Inter-Service Contracts ..... 16
6.5.1. gRPC – GroupMembership Service (Group Service) ..... 16
6.5.2. Events – Redis Streams ..... 17
6.6. Background Jobs ..... 18
7. Technology ..... 18
7.1. Technology Stack ..... 18
7.2. Cross-Cutting Concerns ..... 19
7.3. Deployment – Docker Compose ..... 19
7.4. Requirement Traceability & Bonus Features ..... 20
8. Conceptual ERD ..... 22
9. Physical Database Design ..... 23
9.1. Physical Database Diagrams ..... 23
9.2. Data Dictionary ..... 25
9.2.1. IdentityDb (owned by Identity Service) ..... 25
9.2.2. GroupDb (owned by Group Service) ..... 26
9.2.3. RequestDb (owned by Join Request Service) ..... 29
9.3. Indexes, Constraints & Business-Rule Matrix ..... 30
9.4. Cross-Service Logical References ..... 31
9.5. Migrations & Reference DDL ..... 32
Appendix A. gRPC Contract – group_membership.proto ..... 32
Appendix B. Glossary ..... 33
Appendix C. Generated architecture views ..... 34

# 1. Context

In the EXE (Experiential Entrepreneurship) courses, students must form project groups to develop a startup idea within the rules of their class and semester. Today, teammates are found through fragmented channels – chat groups, spreadsheets and face-to-face talks. With no single source of truth, the head-count of each group, the groups that still have open slots, the pending join requests and the students who still have no group are scattered and hard to track for students, lecturers and administrators alike.

The **EXE Group Management System** replaces these channels with one platform that applies the class rules automatically. It is designed as a distributed .NET system that also demonstrates every PRN232 requirement: RESTful APIs with JWT, a gRPC service, a message broker, background jobs and container-based deployment.

# 2. Problems

- **P1 – Eligible groups are hard to find.** Students cannot easily see which groups of their class match their interests and still have free slots.
- **P2 – Manual request handling and over-capacity.** Team Leaders collect join requests by hand, so requests get lost, and two approvals at the same moment can push a group above the class's maximum size.
- **P3 – Duplicate or invalid memberships.** Without backend validation a student can end up in more than one group of the same EXE class, or in a group of a class they are not enrolled in.
- **P4 – No visibility for lecturers and administrators.** Lecturers cannot monitor grouping progress, under-filled groups or the students of their classes who still have no group.

# 3. Solutions

## 3.1. Proposed Solution

The team builds the EXE Group Management System, a distributed web application that centralizes group formation, join requests and class monitoring:

- **S1 – One workflow for forming groups (P1, P2).** The Single-Page App lets students create groups, search, filter, sort and page through open groups, and submit or cancel join requests; Team Leaders review all requests of their group in one place.
- **S2 – Rules enforced in the backend (P2, P3).** Enrollment and one group per student per class are guaranteed by database constraints (composite foreign keys and unique indexes). The class's MaxMembers and Deadline are checked in the same transaction as every membership change, protected by optimistic concurrency, and a background job locks the groups when the deadline passes.
- **S3 – Class rosters and a monitoring dashboard (P4).** Administrators import the roster of every EXE class; lecturers see grouped and ungrouped students, groups by status and under-filled groups (below MinMembers).
- **S4 – Role- and resource-based access control.** Global roles (Student, Lecturer, Administrator) come from the JWT; the Team Leader permission is checked per group against `Groups.LeaderId` – directly in Group Service and through gRPC from Join Request Service.

- **S5 – Microservices with synchronous and asynchronous communication.** Identity, Group and Join Request services behind a YARP API Gateway, each owning its database; gRPC for immediate checks and the atomic “add member” operation; Redis Streams with a Transactional Outbox and background jobs for reliable events.

## 3.2. Design Rationale – Why Both gRPC and Events

Approving a join request needs an immediate and authoritative answer: is the group still open, is the approver its leader, is there a free slot? Group Service owns that data, so Join Request Service asks it synchronously over gRPC (`GetGroupContext`, `AddMemberIfEligible`) and can reject a request at once with a precise reason.

The outcome of a successful approval is recorded as a `GroupMemberAdded` event in Group Service's outbox, in the same transaction as the new membership, and Join Request Service marks the request **Approved** only when it consumes that event. This gives one reliable path to the final state even when a gRPC reply is lost after the member was added, and the same events drive automatic housekeeping: cancelling the student's other pending requests and rejecting requests to groups that become full, locked or dissolved.

## 3.3. Scope Definition

*Table 4. MVP scope*

| In MVP scope | Out of MVP scope / future extensions |
|---|---|
| JWT sign-in with rotating refresh tokens, role- and resource-based authorization, own profile update. | University SSO / OAuth; server-side revocation of access tokens (sign-out revokes the refresh token; the 60-minute access token simply expires). |
| Administration of users, roles, semesters, courses, classes (MinMembers, MaxMembers, Deadline) and class rosters. | Automatic synchronization with the university timetable / LMS (rosters are imported by the administrator); reopening a closed class. |
| Group create, edit, dissolve, leadership transfer, leave/remove member; search, filter, sort and paginate groups. | AI-based team matching and compatibility scoring. |
| Join-request workflow (submit, cancel, approve, reject) over REST, gRPC and Redis Streams, with automatic cleanup of stale requests. | Real-time chat, video calls and meeting scheduling. |
| Lecturer dashboard; five background jobs; Docker Compose deployment. | E-mail / push notifications – a future Notification service can subscribe to the same group-events stream. |

# 4. Main Actors

## 4.1. Actor Descriptions

*Table 5. Actors*

| Actor | Type | Responsibilities and permissions |
|---|---|---|
| **Student** | Person | Is enrolled in one or more EXE classes. Views own classes, searches groups, and creates a group or submits/cancels join requests in a class where they have no group yet. |
| **Team Leader** | Person (contextual role) | A student who leads a specific group (`Groups. LeaderId`). Edits the group, approves or rejects its join requests, removes members, transfers leadership or dissolves the group. Checked per group – not a JWT role. |
| **Lecturer** | Person | Views assigned classes, grouping progress, under-filled groups and ungrouped students. |
| **Administrator** | Person | Manages user accounts and roles, semesters, courses, classes (lecturer, MinMembers, MaxMembers, Deadline) and class rosters. |

## 4.2. Permission Matrix

*Table 6. Permission matrix*

| Capability | Student | Team Leader | Lecturer | Administrator |
|---|---|---|---|---|
| View, search, filter and sort groups | ✓ [1] | ✓ | ✓ [2] | ✓ |
| Create a group | ✓ [3] | – [4] | – | – |
| Edit group, transfer leadership, dissolve group | – | ✓ (own group) | – | – |
| Remove a member from the group | – | ✓ (own group) | – | – |
| Leave a group | ✓ (member) | – [5] | – | – |
| Submit a join request | ✓ [3] | – [4] | – | – |
| Cancel own pending join request | ✓ | – | – | – |
| View, approve and reject join requests of a group | – | ✓ (own group) | – | – |
| View class overview and ungrouped students | – | – | ✓ [2] | ✓ |
| Manage semesters, courses, classes, limits and deadlines | – | – | – | ✓ |
| Manage class rosters (enrollments) | – | – | – | ✓ |
| Manage user accounts and roles | – | – | – | ✓ |

*[1] Groups of the classes the student is enrolled in. [2] Classes the lecturer is assigned to. [3] Only in a class where the student is enrolled and has no group yet, before the class Deadline. [4] A Team Leader already belongs to a group in that class; in other classes they act as a Student. [5] The leader must transfer leadership first, or dissolve the group when they are its only member.*

# 5. Main Features & Business Rules

## 5.1. Functional Requirements

*Table 7. Functional requirements*

| ID | Feature | Description | Priority | Service |
|---|---|---|---|---|
| FR-01 | JWT authentication | Sign-in issues a 60-minute JWT access token and a 7-day rotating refresh token (POST /api/v1/auth/refresh-token); every service validates the access token (issuer, audience, signing key). Sign-out revokes the refresh token. | Must | Identity |
| FR-02 | Role & resource authorization | Global roles from the JWT; Team Leader checked against `Groups. LeaderId` – locally in Group Service, via gRPC from Join Request Service. | Must | All |
| FR-03 | Group discovery | Search by title/description; filter by class, status and open slots; sort by created date, title or member count; paging with page, pageSize and total count. | Must | Group |
| FR-04 | Group management | Create (creator becomes leader and first member), edit, transfer leadership, dissolve, leave or remove members – before the deadline. | Must | Group |
| FR-05 | Join requests | Submit (Pending, with the requester's code and name as snapshots), cancel own Pending request, list own requests; leaders list the requests of their group. | Must | Join Request |
| FR-06 | Review join requests | Approve (gRPC `AddMemberIfEligible` + confirmation event on Redis Streams) or reject with an optional text. | Must | Join Request, Group |
| FR-07 | Grouping rules | Enrollment and one group per student per class (database constraints); MaxMembers and Deadline (checked in the membership transaction and by the deadline job). | Must | Group |
| FR-08 | Academic configuration | CRUD of semesters, courses and classes (lecturer, MinMembers, MaxMembers, Deadline – rule BR-09); class roster import. | Must | Group |
| FR-09 | Lecturer dashboard | Class overview: grouped and ungrouped students, groups by status, under-filled groups. | Should | Group |
| FR-10 | User & role management | Create, update and lock users; assign roles; update own profile. | Should | Identity |
| FR-11 | Background processing | Outbox publishing, event consumption, deadline enforcement, processing reconciliation and data cleanup (section 6.6). | Must | Group, Join Request |

*Priorities use MoSCoW: Must = required for the MVP and the demo; Should = delivered when time allows. FR-08 and FR-11 are Must because FR-06 and FR-07 depend on class settings and on the background jobs.*

## 5.2. Business Rules

- **BR-01 Enrollment.** A student can create or join a group only in a class whose roster contains them.
- **BR-02 One group per class.** A student belongs to at most one group in each class.
- **BR-03 Leadership.** Every group that is not Dissolved has exactly one leader, who is always one of its members. Leadership can be transferred to another member; the leader can leave only after transferring it. A group can be dissolved only when the leader is its sole member: the membership is removed, the group becomes Dissolved (member count 0, LeaderId kept for history) and GroupClosed (GROUP_DISSOLVED) is published.

- **BR-04 Capacity.** A group never exceeds the class's MaxMembers; it becomes Full at that size and returns to Recruiting when a member leaves or is removed before the deadline. Groups below MinMembers are reported as under-filled.
- **BR-05 Deadline.** After the class Deadline (UTC) no group can be created, joined, left or dissolved and no member can be removed; the deadline job closes the class and locks its groups.
- **BR-06 One active request.** A student has at most one active (Pending or Processing) request to the same group.
- **BR-07 Automatic cleanup.** When a student joins a group, their other Pending requests in that class are cancelled; when a group becomes Full, Locked or Dissolved, its Pending requests are rejected. Only requests created before the triggering event are affected.
- **BR-08 Configurable per class.** MinMembers, MaxMembers and Deadline are set per class by the administrator, never hard-coded.
- **BR-09 Changing class settings.** Lowering MaxMembers below the size of an existing group is rejected (409); otherwise the Full/Recruiting status of every group is recalculated in the same transaction and GroupClosed (GROUP_FULL) is published for groups that become full. The Deadline can be changed only while the class is Open.

## 5.3. Join Request Lifecycle

A join request has five states: **Pending** (waiting for the leader), **Processing** (approved; Group Service is adding the member), **Approved** (membership confirmed by the `GroupMemberAdded` event), **Rejected** (by the leader or by a rule) and **Cancelled** (by the student, or automatically after joining another group).

*Table 8. Join request state transitions*

| # | From | Trigger | Guard | To | Actor |
|---|---|---|---|---|---|
| T1 | – | Submit | Eligible per GetGroupContext (enrolled, no group in the class, group Recruiting, before deadline); no active request to this group | Pending | Student |
| T2 | Pending | Cancel | Caller is the requester | Cancelled | Student |
| T3 | Pending | Reject | Caller is the group's leader (GetGroupContext) | Rejected (REJECTED_BY_LEADER) | Team Leader |
| T4 | Pending | Approve | Caller is the group's leader; conditional update on Status = Pending | Processing | Team Leader |
| T5 | Processing | AddMemberIfEligible returns a rule result (GROUP_FULL, GROUP_LOCKED, GROUP_DISSOLVED, DEADLINE_PASSED, ALREADY_IN_OTHER_GROUP, NOT_ENROLLED, GROUP_NOT_FOUND) | – | Rejected (ReasonCode = result) | System |
| T6 | Processing | GroupMemberAdded consumed for this RequestId | Event not processed before (ProcessedEvents) | Approved | Consumer job |
| T7 | Processing | Transport error, timeout or unexpected status | – | Processing (retried later) | System |

| # | From | Trigger | Guard | To | Actor |
|---|---|---|---|---|---|
| T8 | Pending | GroupMemberAdded for the same student in another group of the class | CreatedAt < event occurredAtUtc | Cancelled (JOINED_ ANOTHER_ GROUP) | Consumer job |
| T9 | Pending | GroupClosed for the group | CreatedAt < event occurredAtUtc | Rejected (GROUP_ FULL / GROUP_ LOCKED / GROUP_ DISSOLVED) | Consumer job |
| T10 | Processing | AddMemberIfEligible returns NOT_ LEADER (leadership changed meanwhile) | – | Pending (DecidedBy, DecidedAt cleared) | System |
| T11 | Processing | AddMemberIfEligible returns ALREADY_ADDED (usually on a ProcessingReconciler retry) | Conditional update on Status = Processing | Approved; the student's other Pending requests in the class created before added_at_utc → Cancelled (JOINED_ANOTHER_ GROUP), same confirmation as T6 + T8 | System |

Every transition is a conditional update (`WHERE Status = <expected>`) protected by `RowVersion`; if no row is affected the API returns 409 Conflict. Cancel is allowed only from Pending, so a request cannot be cancelled while Group Service is adding the member. One list of reason codes is shared by the gRPC results, the events and `JoinRequests.ReasonCode`:

*Table 9. Reason codes*

| Reason code | Meaning |
|---|---|
| `GROUP_ FULL` | The group reached the class's MaxMembers. |
| `GROUP_ LOCKED` | The class deadline passed and the group was locked. |
| `GROUP_ DISSOLVED` | The group was dissolved by its leader. |
| `DEADLINE_ PASSED` | The class deadline passed before the member could be added. |
| `ALREADY_ IN_ OTHER_ GROUP` | The student already belongs to another group of the class. |
| `NOT_ ENROLLED` | The student is not on the class roster. |
| `GROUP_ NOT_ FOUND` | The group does not exist. |
| `JOINED_ ANOTHER_ GROUP` | Automatic cancellation: the student joined another group of the class. |
| `REJECTED_ BY_ LEADER` | The Team Leader rejected the request (optional text in ReasonText). |

## 5.4. RESTful API Endpoints

### 5.4.1. Conventions

- Resource-oriented, plural, kebab-case URLs under `/api`; JSON bodies; every route passes through the API Gateway (the web container proxies `/api` to it, so the browser stays on one origin).
- Authorization is expressed with roles and policies (e.g. `[Authorize(Roles = "Administrator")]`, resource checks), not with URL prefixes.
- List endpoints accept search, sort (comma-separated fields, '-' for descending, e.g. sort=-createdAt,title), page, pageSize (default 20, values above 100 become 100), fields (field selection) and expand; the envelope carries pagination { page, pageSize, totalItems, totalPages }.
- Status codes: 200/201 (with Location)/202/204 on success, 400 validation, 401/403 authentication/authorization, 404 not found, 406 unsupported Accept, 409 business-rule or

concurrency conflict. Every response – success or error, including the gateway's own 401/429/502 – uses one envelope { success, message, data, errors: [{ code, field, message }], traceId, pagination? } (JSON, or XML when the client asks for application/xml).

- Approve returns 202 Accepted with `Location: /api/v1/join-requests/{id}` and data { requestId, `status: "Processing"` }; the Single-Page App polls that resource every 1–2 s (up to about 30 s) until it is Approved or Rejected.
- Join-request state changes are explicit actions (`POST /{id}/approve`, `/reject`, `/cancel`) because they are state transitions, not field edits.
- Transport security: in production TLS terminates at nginx and the gateway; the local Docker Compose demo uses plain HTTP.

### 5.4.2. Identity Service

*Table 10. Identity Service endpoints*

| Method | Endpoint | Role | Purpose |
|---|---|---|---|
| POST | `/ api/ v1/ auth/ login` | Anonymous | Verify credentials and issue a JWT access token (60 min) and a refresh token (7 days). |
| POST | `/api/v1/auth/refresh-token` | Anonymous | Rotate a refresh token: revoke it and issue a new access and refresh token pair. |
| POST | `/api/v1/auth/logout` | Anonymous | Revoke the given refresh token; 204, also when it is already revoked or unknown. |
| GET | `/ api/ v1/ auth/ me` | Any signed-in user | Current profile and roles. |
| PUT | `/ api/ v1/ auth/ me` | Any signed-in user | Update own profile (full name). |
| GET | `/ api/ v1/ users` | Administrator | Search users; filter by `role` and `status`; sort and paging. |
| GET | `/ api/ v1/ users/{id}` | Administrator | User details. |
| POST | `/ api/ v1/ users` | Administrator | Create an account with roles (Student accounts require a user code). |
| PUT | `/ api/ v1/ users/{id}` | Administrator | Update e-mail, user code and full name. |
| PATCH | `/ api/ v1/ users/{id}/ status` | Administrator | Lock or unlock an account (accounts are deactivated, not deleted). |
| PUT | `/ api/ v1/ users/{id}/ roles` | Administrator | Replace the role set of a user. |

### 5.4.3. Group Service

*Table 11. Group Service endpoints*

| Method | Endpoint | Role | Purpose |
|---|---|---|---|
| GET | `/ api/ v1/ semesters` | Any signed-in user | List semesters (paged, sortable). |
| POST / PUT / DELETE | `/ api/ v1/ semesters[/{id}]` | Administrator | Create, update, delete a semester (delete only when it has no classes). |
| GET | `/ api/ v1/ courses` | Any signed-in user | List courses (paged, sortable). |
| POST / PUT / DELETE | `/ api/ v1/ courses[/{id}]` | Administrator | Create, update, delete a course (delete only when unused). |
| GET | `/ api/ v1/ classes` | Any signed-in user | Filter by `semesterId`, `courseId`, `search`; `mine=true` returns the classes the caller is enrolled in (with `myGroupId` and `isLeader`) or teaches. |
| GET | `/ api/ v1/ classes/{id}` | Any signed-in user | Class details with MinMembers, MaxMembers, Deadline, Status. |

| Method | Endpoint | Role | Purpose |
|---|---|---|---|
| POST / PUT / DELETE | `/api/v1/classes[/{id}]` | Administrator | Create; update lecturer, limits and deadline (rule BR-09); delete only when the class has no groups – its roster is deleted in the same transaction. |
| GET | `/api/v1/classes/{id}/enrollments` | Lecturer (assigned), Administrator | Class roster; `grouped=false` lists the ungrouped students; search and paging. |
| POST | `/api/v1/classes/{id}/enrollments` | Administrator | Enroll or refresh a batch of students picked from the user list (studentId plus code, name and e-mail snapshots; upsert). |
| DELETE | `/api/v1/classes/{id}/enrollments/{studentId}` | Administrator | Remove a student who is not in a group. |
| GET | `/api/v1/classes/{id}/overview` | Lecturer (assigned), Administrator | Grouping statistics: grouped / ungrouped counts, groups by status, under-filled groups. |
| GET | `/api/v1/groups` | Any signed-in user (scoped) | Search groups of the caller's classes (Students: enrolled, Lecturers: assigned, Administrators: all); filter by `classId`, `status`, `hasOpenSlots`; sort by `createdAt`, `title` or `memberCount`; paging. |
| GET | `/api/v1/groups/{id}` | Any signed-in user (scoped) | Group details with member list (names from the roster snapshot). |
| POST | `/api/v1/groups` | Student (eligible) | Create a group; the creator becomes Team Leader and first member. |
| PUT | `/api/v1/groups/{id}` | Team Leader | Edit title and description. |
| DELETE | `/api/v1/groups/{id}` | Team Leader | Dissolve the group: only when the leader is the sole member, before the deadline. |
| PUT | `/api/v1/groups/{id}/leader` | Team Leader | Transfer leadership to another member. |
| DELETE | `/api/v1/groups/{id}/members/{studentId}` | Member (self), Team Leader | Leave the group or remove a member, before the deadline; the leader must transfer leadership first. |

### 5.4.4. Join Request Service

*Table 12. Join Request Service endpoints*

| Method | Endpoint | Role | Purpose |
|---|---|---|---|
| GET | `/api/v1/join-requests` | Student; Team Leader | `mine=true` returns own requests (with group title); `groupId={id}` returns the requests of the leader's group (with the requester's code and name); filter by `status`; paging. |
| GET | `/api/v1/join-requests/{id}` | Requester, Team Leader | Request details and current status (used for polling after approve). |
| POST | `/api/v1/join-requests` | Student (eligible) | Submit a request to a group → 201, status Pending. |
| POST | `/api/v1/join-requests/{id}/cancel` | Requester | Cancel a Pending request → Cancelled. |
| POST | `/api/v1/join-requests/{id}/approve` | Team Leader | Approve → 202 Accepted with `Location: /api/v1/join-requests/{id}` and `{ requestId, status: Processing }`; 409 with the rule that failed. |
| POST | `/api/v1/join-requests/{id}/reject` | Team Leader | Reject with an optional text → Rejected (REJECTED_BY_LEADER). |

# 6. System Architecture – C4 Model (Level 1 & Level 2)

The architecture is described with the C4 model (c4model.com). Level 1 (System Context) and Level 2 (Container) are drawn below; the Level 3 component views of the three services, the Docker Compose deployment view and the approve-flow dynamic view are generated from the model in docs/architecture/workspace.dsl (Structurizr DSL) and shown in Appendix C. Level 4 (code) is out of scope. Both diagrams below read from left to right and use the same people and relationship meanings, so Figure 2 is a direct zoom-in of Figure 1.

## 6.1. Level 1: System Context Diagram

Three kinds of people use the system. In the MVP the system has no external software-system dependencies – university SSO and LMS/timetable synchronization are out of scope (section 3.3) – so no external systems appear.

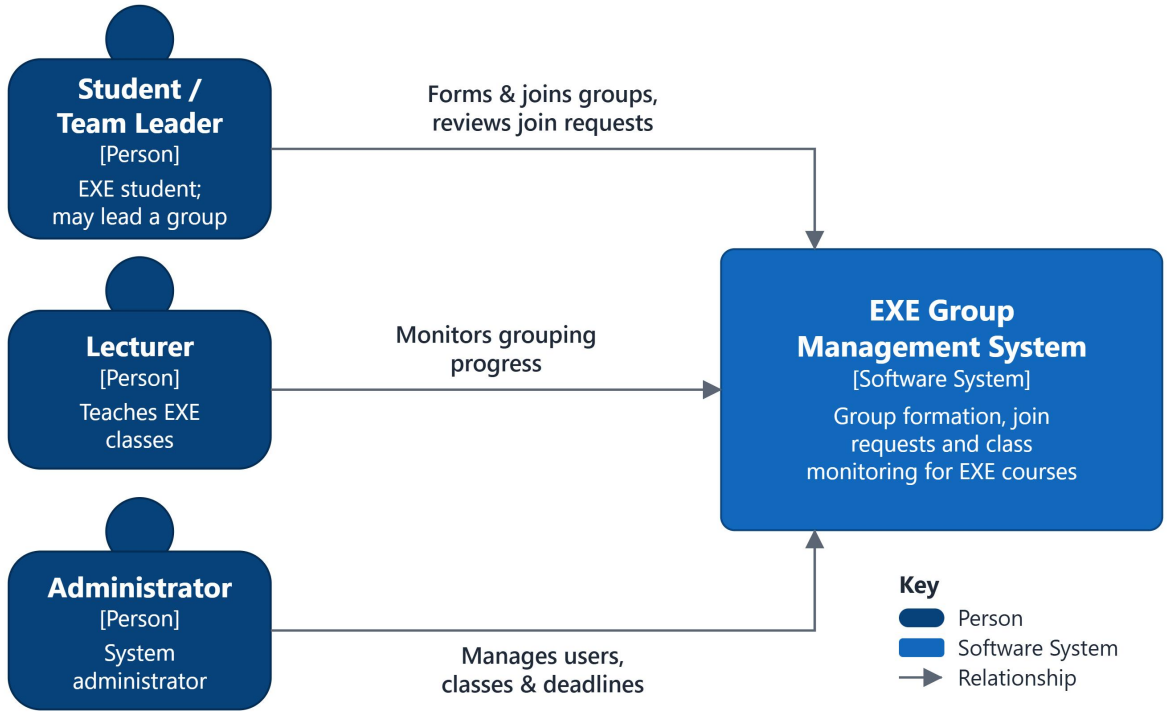



*Figure 1. C4 Level 1 – System Context diagram for EXE Group Management System*

## 6.2. Level 2: Container Diagram

Requests flow from left to right. Solid arrows are synchronous calls; dashed arrows are events through the Group Events stream, which is modeled as a container as c4model.com recommends (not the whole message bus). The nginx web container, which serves the Single-Page App files and forwards `/api` calls to the gateway (section 7.3), is omitted for compactness.

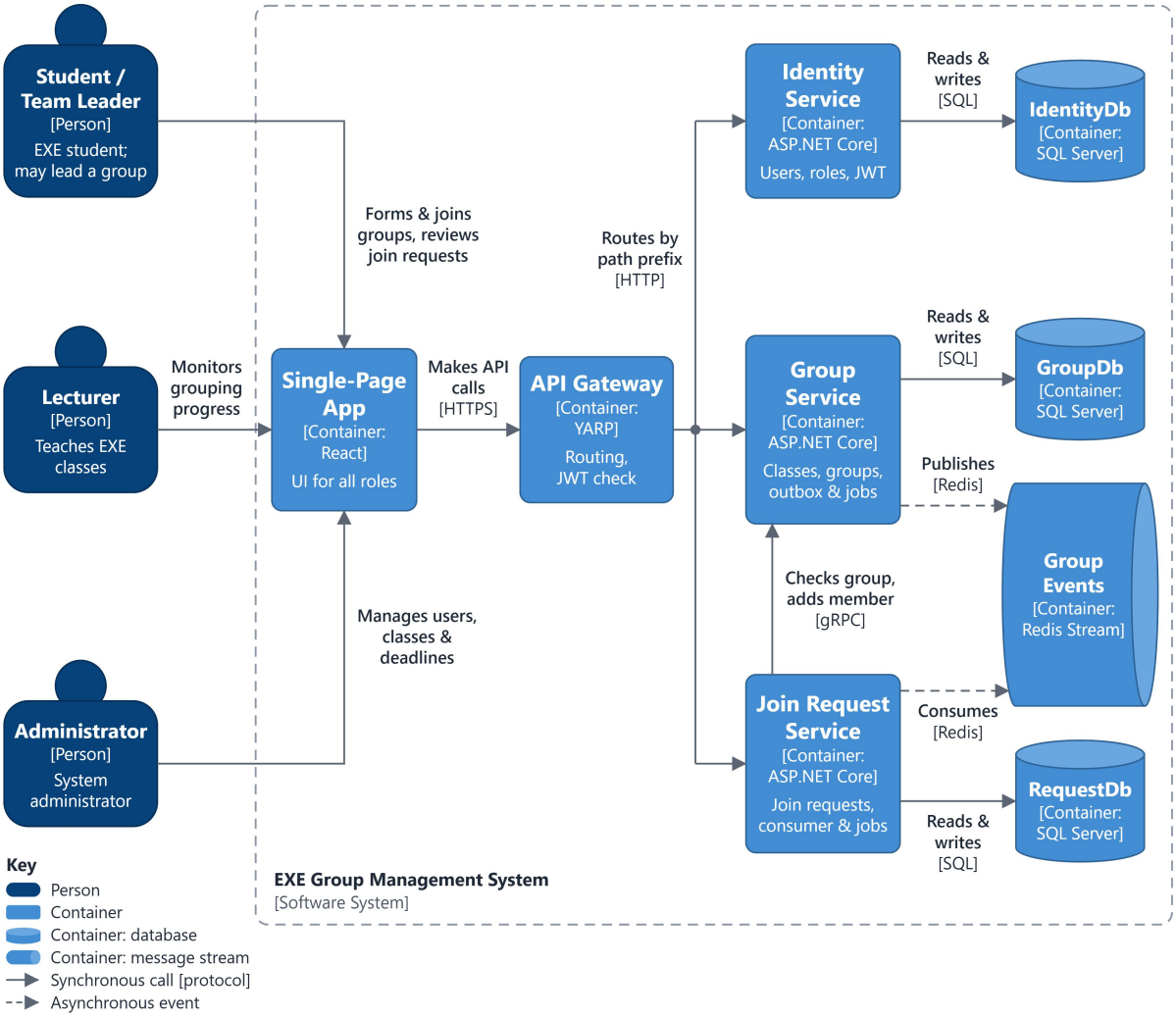



*Figure 2. C4 Level 2 – Container diagram for EXE Group Management System*

## 6.3. Container Responsibilities

*Table 13. Container responsibilities*

| Container | Technology | Responsibilities |
|---|---|---|
| **Single-Page App** | React 19, TypeScript (Vite), served by nginx | UI for students, Team Leaders, lecturers and administrators; talks only to the API Gateway. |
| **API Gateway** | ASP.NET Core 10 + YARP | Single public API entry point; validates the JWT at the edge; routes by path prefix (section 7.3); forwards the Authorization and X-Correlation-ID headers; serves the Swagger UI. |
| **Identity Service** | ASP.NET Core 10 Web API | Sign-in and JWT issuing, user accounts, roles, own profile. |

| Container | Technology | Responsibilities |
|---|---|---|
| **Group Service** | ASP.NET Core 10 Web API + gRPC server | Semesters, courses, classes, rosters, groups and memberships; gRPC GroupMembership service; OutboxPublisher, DeadlineEnforcer and DataCleanup jobs. |
| **Join Request Service** | ASP.NET Core 10 Web API + gRPC client | Join-request lifecycle; calls Group Service over gRPC; GroupEventsConsumer, ProcessingReconciler and DataCleanup jobs. |
| **IdentityDb** | SQL Server 2022 | Users, roles and their assignments (Identity Service only). |
| **GroupDb** | SQL Server 2022 | Semesters, courses, classes, rosters, groups, memberships, idempotency records and the outbox (Group Service only). |
| **RequestDb** | SQL Server 2022 | Join requests and processed event ids (Join Request Service only). |
| **Group Events** | Redis 7 Stream (key `group-events`) | Carries GroupMemberAdded and GroupClosed from Group Service (producer) to Join Request Service (consumer group `join-request-service`). |

Each database is changed only by its owning service through EF Core migrations; there are no cross-database foreign keys. Display names are snapshots taken when data is written – ClassEnrollments for rosters and member lists, JoinRequests for the requester's code and name – so no service reads another service's database to render a list.

## 6.4. Inter-Service Workflow – Approving a Join Request

1. The Team Leader clicks Approve; the Single-Page App calls `POST /api/v1/join-requests/{id}/approve` through the gateway [REST].
2. Join Request Service calls `GetGroupContext` [gRPC] and checks that the caller is the group's leader (otherwise 403; nothing changes).
3. It moves the request Pending → Processing with a conditional update and records DecidedBy/DecidedAt (409 if the request changed meanwhile).
4. It calls `AddMemberIfEligible(requestId, groupId, studentId, approverId)` [gRPC] with a 5-second deadline.
5. Group Service first checks idempotency: a request_id already in AddMemberRequests, or a student already in this group, returns ALREADY_ADDED; a student in another group of the class returns ALREADY_IN_OTHER_GROUP. Otherwise, in one transaction, it re-checks leader, enrollment, deadline, group status and capacity (RowVersion), inserts the GroupMembers and AddMemberRequests rows, updates CurrentMemberCount (Status = Full at the limit) and writes `GroupMemberAdded` – plus `GroupClosed (GROUP_FULL)` if the group became full – to OutboxEvents.
6. A rule result sets the request to Rejected with its ReasonCode (409 to the leader); NOT_LEADER returns it to Pending (403); success answers 202 Accepted (Processing).
7. OutboxPublisher publishes the events to the `group-events` stream within about 2 seconds and marks them published.
8. GroupEventsConsumer stores the EventId in ProcessedEvents and, in the same transaction, sets the request to Approved and cancels the student's other Pending requests in that class created before the event; it then acknowledges the message (XACK).
9. If no event arrives – for example because the gRPC call timed out – ProcessingReconciler repeats `AddMemberIfEligible` for requests still Processing 2 minutes after DecidedAt; the call is idempotent by request_id, so a member is never added twice. When the retry returns

ALREADY_ADDED, that reply confirms the request as Approved (T11), so a lost event cannot leave it in Processing.

Other flows use the same building blocks:

- **Submit a request** – Join Request Service calls `GetGroupContext` to check enrollment, no existing group in the class, group status and deadline, stores ClassId, StudentCode, StudentName and GroupTitle, and inserts the request as Pending (201).
- **Create a group** – Group Service inserts the group and the leader's membership and publishes `GroupMemberAdded` (requestId = null), which cancels the creator's Pending requests in that class.
- **Leave or remove a member** – Group Service deletes the membership before the deadline, decrements CurrentMemberCount and turns a Full group back to Recruiting; no event is needed.
- **Dissolve a group** – allowed when the leader is the sole member: the membership is deleted, the group becomes Dissolved with count 0, and `GroupClosed (GROUP_DISSOLVED)` rejects its Pending requests.
- **Deadline passes** – DeadlineEnforcer closes the class and locks its groups; the `GroupClosed (GROUP_LOCKED)` events make the consumer reject their Pending requests.

## 6.5. Inter-Service Contracts

### 6.5.1. gRPC – GroupMembership Service (Group Service)

*Table 14. gRPC methods (messages <Method>Request and <Method>Reply)*

| Method | Request fields | Reply fields | Used for |
|---|---|---|---|
| `GetGroupContext` | group_ id, student_ id | exists, class_ id, leader_ id, group_ status, deadline_ utc, current_ member_ count, max_ members, group_ title, student_ enrolled, student_ has_ group_ in_ class, student_ code, student_ full_ name | Read-only. Submit (eligibility and snapshots); list, approve and reject (is the caller the leader?). |
| `AddMemberIfEligible` | request_ id, group_ id, student_ id, approver_ id | result, membership_ id, current_ member_ count, added_ at_ utc | Approve and ProcessingReconciler. Idempotent by request_ id; adds the member, the idempotency record and the outbox event in one transaction. |

*Table 15. AddMemberIfEligible results (prefixed ADD_MEMBER_RESULT_ in the .proto)*

| Result | Meaning | Effect on the join request |
|---|---|---|
| `ADDED` | Member added; GroupMemberAdded written to the outbox. | Stays Processing until the event is consumed (202). |
| `ALREADY_ ADDED` | request_ id was already processed, or the student is already in this group – checked first, before any rule; no second event. | Approved at once (T11): the reply is authoritative, so a lost GroupMemberAdded event cannot leave the request in Processing. |
| `GROUP_ FULL` | CurrentMemberCount reached MaxMembers. | Rejected, ReasonCode GROUP_ FULL (409). |
| `GROUP_ LOCKED` | The class deadline passed and the group is Locked. | Rejected, ReasonCode GROUP_ LOCKED (409). |
| `GROUP_ DISSOLVED` | The group was dissolved. | Rejected, ReasonCode GROUP_ DISSOLVED (409). |
| `DEADLINE_ PASSED` | The class deadline has passed (group not locked yet). | Rejected, ReasonCode DEADLINE_ PASSED (409). |

| Result | Meaning | Effect on the join request |
|---|---|---|
| ALREADY_ IN_ OTHER_ GROUP | The student already belongs to another group of the class. | Rejected, ReasonCode ALREADY_ IN_ OTHER_ GROUP (409). |
| NOT_ ENROLLED | The student is not on the class roster. | Rejected, ReasonCode NOT_ ENROLLED (409). |
| GROUP_ NOT_ FOUND | The group does not exist. | Rejected, ReasonCode GROUP_ NOT_ FOUND (409). |
| NOT_ LEADER | approver_ id is no longer the group's leader. | Back to Pending (T10); 403 to an interactive caller, logged for the reconciler. |
| Transport error / timeout | UNAVAILABLE, DEADLINE_ EXCEEDED, ABORTED (5 s client deadline). | Stays Processing; ProcessingReconciler retries (202). |
| Other status / UNSPECIFIED | INTERNAL, UNKNOWN, INVALID_ ARGUMENT or result value 0. | Stays Processing, error logged; ProcessingReconciler retries. |

The complete `group_membership.proto` is listed in Appendix A. Business outcomes are returned in the `result` field; gRPC status codes are reserved for transport problems. The gRPC endpoint (port 8081) is service-to-service only: no end-user JWT is forwarded, the trust boundary is the internal Docker network, and `approver_id` is the `sub` already validated by Join Request Service.

### 6.5.2. Events – Redis Streams

Group Service appends events to the stream `group-events` (`XADD … MAXLEN ~ 10000`). At start-up Join Request Service creates the consumer group `join-request-service` (`XGROUP CREATE … MKSTREAM`, ignoring BUSYGROUP) and then reads in a polling loop with `XREADGROUP` – StackExchange.Redis deliberately offers no blocking reads. Each message is acknowledged (`XACK`) after its database transaction commits, and messages left pending by a crashed consumer are re-claimed with `XAUTOCLAIM`. Delivery is at-least-once; duplicates are discarded through ProcessedEvents. Every message has the envelope fields `eventId`, `eventType`, `occurredAtUtc`, `correlationId` and `payload` (JSON).

*Table 16. Events on the group-events stream*

| Event | Meaning | Payload | Consumer effect |
|---|---|---|---|
| GroupMemberAdded | A student became a member of a group (approval or group creation). | groupId, classId, studentId, requestId (null when the leader creates the group), currentMemberCount | Request → Approved (if Processing); the student's other Pending requests in the class created before occurredAtUtc → Cancelled (JOINED_ ANOTHER_ GROUP). |
| GroupClosed | A group stopped accepting members. | groupId, classId, reason (GROUP_ FULL \| GROUP_ LOCKED \| GROUP_ DISSOLVED) | Pending requests of the group created before occurredAtUtc → Rejected with the reason. |

## 6.6. Background Jobs

*Table 17. Background jobs (BackgroundService + PeriodicTimer)*

| Job | Service | Schedule | Responsibility |
|---|---|---|---|
| **OutboxPublisher** | Group Service | Every 2 s | Reads unpublished OutboxEvents (oldest first), XADDs them to group-events, sets PublishedAtUtc; on failure increments RetryCount and stores LastError. |
| **DeadlineEnforcer** | Group Service | Every 1 min | Closes classes whose Deadline has passed, locks their Recruiting/ Full groups and writes GroupClosed (GROUP_ LOCKED) to the outbox. |
| **GroupEventsConsumer** | Join Request Service | Polling loop (0.5–1 s pause when idle) | Creates the consumer group at start-up (XGROUP CREATE … MKSTREAM); reads with XREADGROUP COUNT 50 (StackExchange.Redis has no blocking reads); applies events idempotently and XACKs after commit; every 30 s re-claims messages idle for 60 s with XAUTOCLAIM. |
| **ProcessingReconciler** | Join Request Service | Every 1 min | Calls AddMemberIfEligible again for requests still Processing 2 minutes after DecidedAt; safe because the call is idempotent by request_ id. ALREADY_ADDED confirms the request as Approved (T11), so it never waits for an event that was lost. |
| **DataCleanup** | Group Service and Join Request Service | Daily, 02:00 UTC | Deletes OutboxEvents published more than 7 days ago and AddMemberRequests and ProcessedEvents older than 30 days; the stream itself is capped by XADD MAXLEN ~ 10000. |

# 7. Technology

## 7.1. Technology Stack

*Table 18. Technology stack*

| Layer / capability | Technology | Purpose and PRN232 alignment |
|---|---|---|
| **Backend framework** | ASP.NET Core 10 (.NET 10 LTS) | Three projects per service (API → Services → Repositories), enforced by architecture tests, dependency injection, Options-based configuration, logging, global exception handling. |
| **API Gateway** | YARP 2.x | Single entry point, edge JWT validation, path-based routing, Swagger UI. |
| **Persistence** | EF Core 10, SQL Server 2022 | Database per service, Code-First migrations, rowversion concurrency, filtered unique indexes, CHECK constraints. |
| **Synchronous communication** | gRPC: Grpc.AspNetCore, Grpc.Net.Client, Protocol Buffers over HTTP/2 | GroupMembership service: GetGroupContext, AddMemberIfEligible (section 6.5.1). |
| **Message broker** | Redis 7 Streams via StackExchange.Redis 2.7+ | Producer Group Service, consumer group Join Request Service; XADD, XREADGROUP (polling), XACK, XAUTOCLAIM. |
| **Background processing** | .NET BackgroundService + PeriodicTimer | Five jobs (section 6.6). |
| **Security** | JWT Bearer (HS256, key from configuration); ASP.NET Core PasswordHasher (PBKDF2) | Authentication in every service; role- and resource-based authorization. RS256 with a JWKS endpoint is a later hardening step. |
| **API documentation** | Swashbuckle (Swagger / OpenAPI 3) | One OpenAPI document per service, all shown in the gateway's Swagger UI at `/ swagger`. |
| **Frontend** | React 19, TypeScript, Vite, nginx | Single-Page App that talks only to the gateway. |

| Layer / capability | Technology | Purpose and PRN232 alignment |
|---|---|---|
| **Deployment** | Docker, Docker Compose | Seven containers (section 7.3). |

**Broker decision – Redis Streams.** PRN232 allows Apache Kafka or Redis. Redis Streams was chosen because it runs as one lightweight container and still offers persistent streams, consumer groups, acknowledgements and a pending-entries list for redelivery – the at-least-once delivery that the outbox and the idempotent consumer rely on. Redis Pub/Sub was rejected because it does not store messages (an offline consumer loses them). Kafka would also fit, but adds a heavier JVM-based broker without a benefit at this scale.

## 7.2. Cross-Cutting Concerns

- **Configuration:** `appsettings.json` per service, overridden by environment variables from Docker Compose and a git-ignored `.env` file (e.g. `ConnectionStrings__GroupDb`, `Jwt__SigningKey`, `Redis__Connection`, `GrpcClients__GroupService`); no secrets in source control.
- **Logging and tracing:** structured logging with Serilog; the gateway creates an `X-Correlation-ID` that travels in HTTP headers, gRPC metadata and the event envelope, so one approval can be traced across all services.
- **Exception handling: global middleware converts exceptions to the response envelope with a business code (validation 400, missing or invalid token 401, forbidden 403, not found 404, business rule or concurrency conflict 409, anything else 500 without internal details); status-code pages give 404/405/406/415/429 the same envelope.**
- **Security:** only the web (3000) and gateway (8080) containers publish ports – SQL Server 1433 only in development; every service validates the JWT (same issuer, audience and key) and takes the caller from the `sub` claim; gRPC (8081) is reachable only on the internal Docker network.
- **Data access:** one DbContext and repositories per service; sequential GUID keys generated by EF Core (known before SaveChanges, less index fragmentation); all timestamps in UTC.
- **Health checks and seed data:** Docker Compose uses `/health/live` and `/health/ready`; readiness checks the service's own database, plus Redis for Group Service and Join Request Service (the gateway reports liveness only). In Development, Identity Service seeds the three roles and a first Administrator account.

## 7.3. Deployment – Docker Compose

*Table 19. Docker Compose services*

| Service | Image / build | Ports | Depends on | Notes |
|---|---|---|---|---|
| **web** | nginx:1.27-alpine + React build | 3000 → 80 | gateway | Serves the Single-Page App and reverse-proxies /api and /swagger to the gateway (same origin, so no CORS). |
| **gateway** | Build: src/ApiGateway (YARP) | 8080 → 8080 | identity-service, group-service, join-request-service | Single public API entry point; validates the JWT; routes by path prefix; Swagger UI. |
| **identity-service** | Build: src/IdentityService | internal 8080 | sqlserver | REST; owns IdentityDb. |
| **group-service** | Build: src/GroupService | internal 8080 (REST), 8081 (gRPC, HTTP/2 only) | sqlserver, redis | REST + gRPC server; outbox, deadline and cleanup jobs. |

| Service | Image / build | Ports | Depends on | Notes |
|---|---|---|---|---|
| **join-request-service** | Build: src/JoinRequestService | internal 8080 | sqlserver, redis, group-service | REST; gRPC client of http://group-service:8081; consumer, reconciler and cleanup jobs. |
| **sqlserver** | mcr.microsoft.com/mssql/server:2022-latest | 1433 (development only) | – | One instance hosting IdentityDb, GroupDb and RequestDb; data volume. |
| **redis** | redis:7-alpine (appendonly yes) | internal 6379 | – | Hosts the group-events stream; data volume. |

*Table 20. API Gateway routes (YARP)*

| Path prefix | Destination |
|---|---|
| `/api/v1/auth/login, /api/v1/auth/refresh-token, /api/v1/auth/logout` | identity-service – anonymous (no JWT required), rate limited |
| `/ api/ v1/ auth/**, / api/ v1/ users/**` | identity-service |
| `/ api/ v1/ semesters/**, / api/ v1/ courses/**, / api/ v1/ classes/**, / api/ v1/ groups/**` | group-service |
| `/ api/ v1/ join-requests/**` | join-request-service |
| `/ swagger/{identity\|group\|join-request}/**` | OpenAPI document of the matching service (prefix removed); the Swagger UI at /swagger lists all three |

Group Service listens on two Kestrel endpoints: 8080 (HTTP/1.1, REST) and 8081 (HTTP/2 only, gRPC), because gRPC without TLS (h2c) cannot share an HTTP/1.1 endpoint. EF Core migrations are applied at start-up in Development; elsewhere the team runs `dotnet ef database update` or `database/schema.sql`. Running `docker compose up -d` starts the whole system; the Single-Page App is at http://localhost:3000 and the Swagger UI at http://localhost:8080/swagger.

## 7.4. Requirement Traceability & Bonus Features

*Table 21. PRN232 requirement traceability*

| Assignment section (PRN232 – Final Assignment) | How the design satisfies it | Demo step |
|---|---|---|
| §2.1 REST API: CRUD, layered architecture, JWT, search / filter / sort / paging | Three services behind a gateway (section 5.4); Controllers → Services → Repositories in every service (sections 7.1–7.2); FR-01 to FR-10 | Sign in; CRUD a class; search groups with paging and sorting in Swagger |
| §2.2 Background job | Five jobs (section 6.6) | Approve a request and show OutboxPublisher and consumer logs; pass a deadline and show the groups locked |
| §2.3 Message broker: producer and consumer | Redis Streams `group-events`; producer Group Service (outbox), consumer Join Request Service (section 6.5.2) | Show the stream entries (XRANGE / RedisInsight) and the request turning Approved |
| §2.4 gRPC service and REST ↔ gRPC interaction | GroupMembership gRPC server in the independent group-service container (HTTP/2 port 8081); the Join Request Service REST API calls it on submit, list by group, approve and reject (sections 6.4–6.5.1) | Submit and approve a request; follow the gRPC calls by correlation id |
| §3 Deployment | Docker Compose with seven containers (section 7.3) | `docker compose up`; all health checks green |
| §4 Technical requirements | .NET 10, EF Core, SQL Server, DI, configuration, logging, exception handling, JWT, Swagger, Docker (sections 7.1–7.2) | Swagger UI; error envelope with a business code on a rule violation |
| §5 Deliverables | EF Core migrations + reference DDL `database/ schema. sql`, Dockerfiles, compose file, Swagger, README | – |

| Assignment section (PRN232 – Final Assignment) | How the design satisfies it | Demo step |
|---|---|---|
| §6 Demonstration – end-to-end workflow | Approve flow (section 6.4) | Student submits → leader approves → member added → other requests cancelled |

**Bonus features beyond the minimum requirements:**

- API Gateway with YARP (single entry point, edge authentication, Swagger UI).
- Transactional Outbox, idempotent consumer (ProcessedEvents) and idempotent gRPC operation (AddMemberRequests) – an exactly-once effect on top of at-least-once delivery.
- Optimistic concurrency with rowversion for the last free slot and for request state transitions.
- Business rules enforced in the database – enrollment and class consistency through composite foreign keys, one group per class and one active request through (filtered) unique indexes, value checks through CHECK constraints – verified by running the DDL and 34 tests on SQL Server 2019.
- ProcessingReconciler and DataCleanup jobs, correlation-id tracing and health checks.

# 8. Conceptual ERD

Figure 3 shows the nine domain entities, colored by the bounded context (and database) that owns them. Solid lines are relationships inside one database; dashed lines are references between services that are maintained by the services, not by foreign keys. Cardinalities use crow's-foot notation, explained in the key under the figure. Attributes and data types belong to the physical design (section 9).

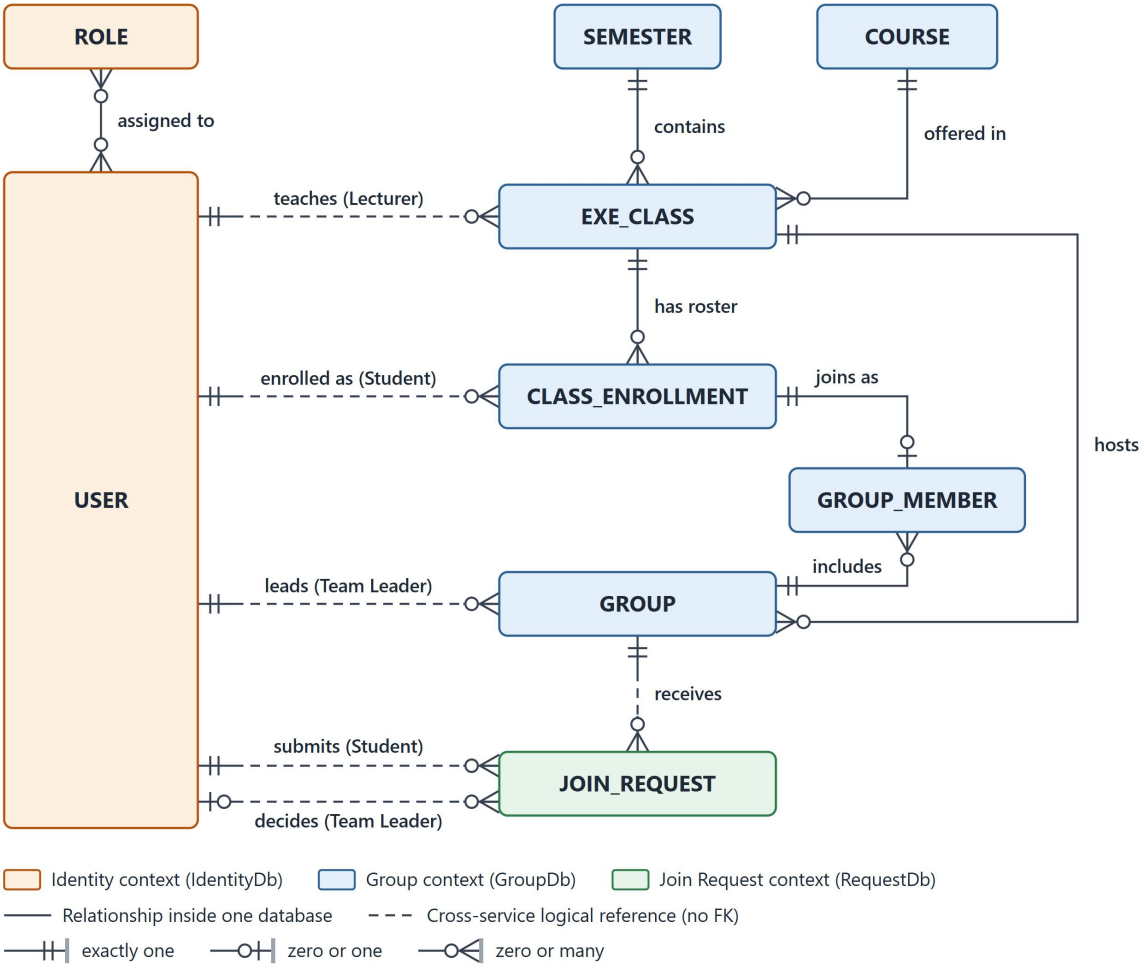



*Figure 3. Conceptual ERD*

- A USER can have several ROLEs (many-to-many).
- A SEMESTER contains, and a COURSE is offered in, many EXE_CLASSes; each EXE_CLASS is taught by one Lecturer (USER).
- CLASS_ENROLLMENT is the roster of an EXE_CLASS and links each enrolled Student (USER) to it.
- An enrolled student joins at most one GROUP of the class: each CLASS_ENROLLMENT has at most one GROUP_MEMBER (BR-02). A GROUP has at least one member – its Team Leader – until it is dissolved (BR-03).
- A GROUP receives JOIN_REQUESTs; a Student submits them and the group's Team Leader decides them.

# 9. Physical Database Design

Each service owns one database on a shared SQL Server 2022 instance: **IdentityDb**, **GroupDb** and **RequestDb**. The schemas are created by EF Core Code-First migrations; the equivalent reference DDL is delivered in `database/schema.sql`.

## 9.1. Physical Database Diagrams

Figures 4–6 show one diagram per database, in the same order as section 9.2 and at the same scale. Connectors are foreign keys in crow's-foot notation, labeled with the foreign-key columns. Columns marked LR refer to another service's database without a foreign key (section 9.4); complete types, defaults and constraints are listed in section 9.2.

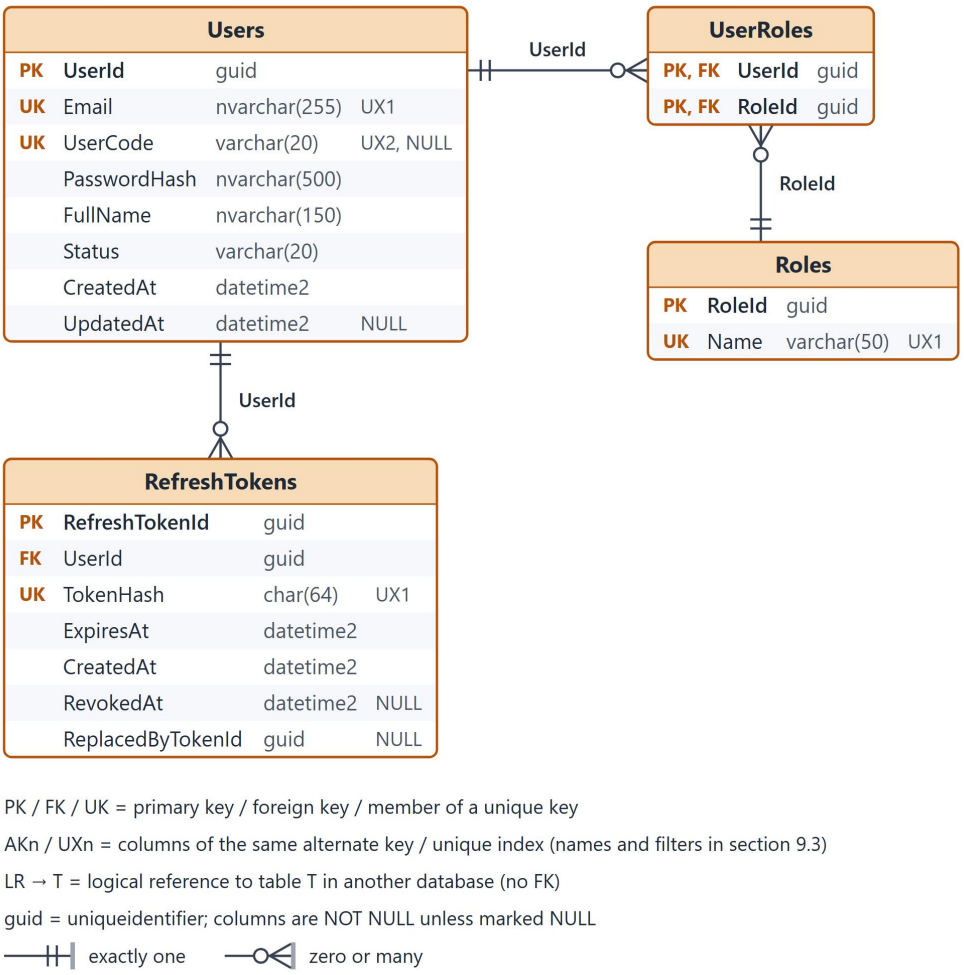
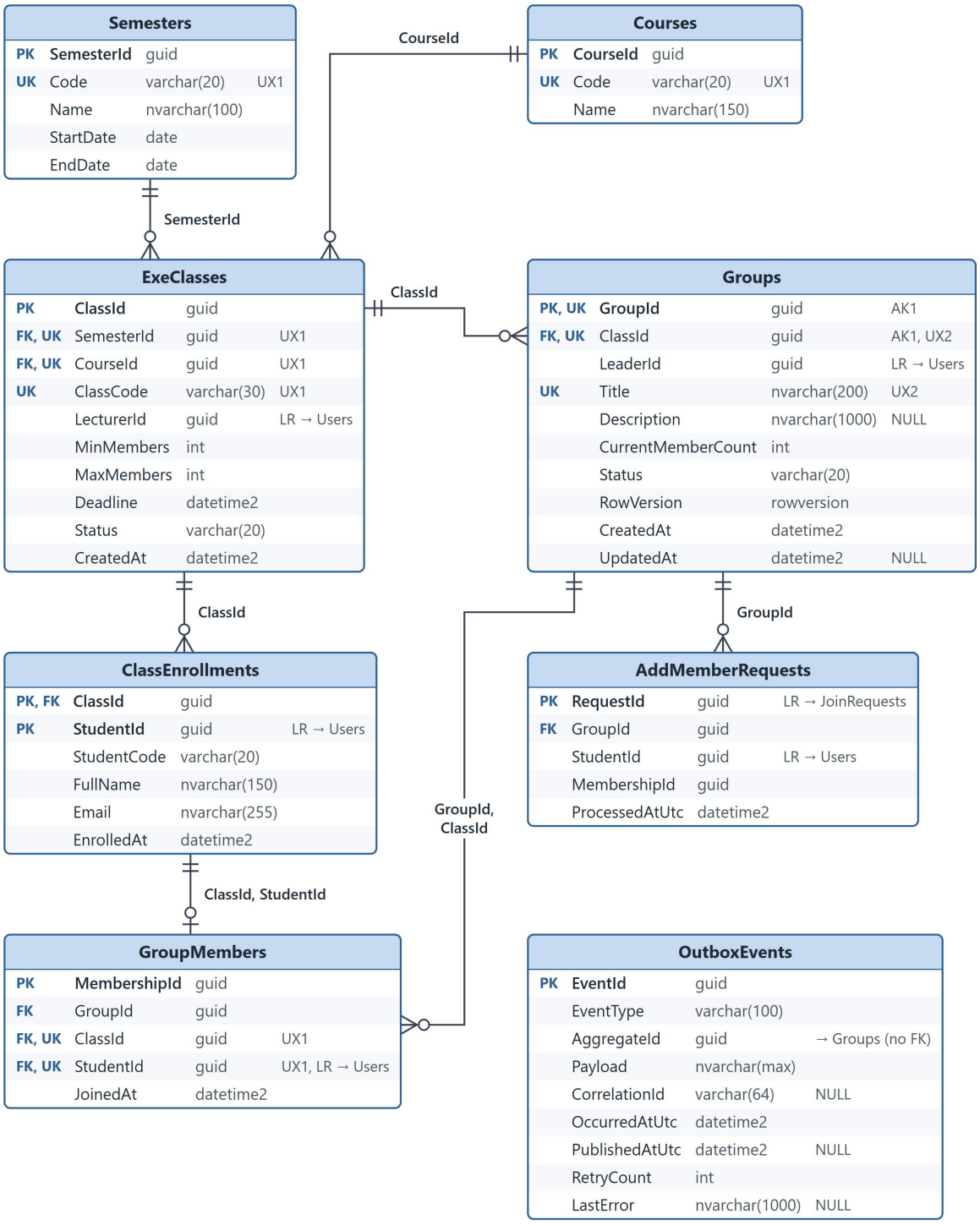



*Figure 4. IdentityDb (Identity Service)*




PK / FK / UK = primary key / foreign key / member of a unique key

AKn / UXn = columns of the same alternate key / unique index (names and filters in section 9.3)

LR → T = logical reference to table T in another database (no FK)

guid = uniqueidentifier; columns are NOT NULL unless marked NULL

exactly one  zero or one  zero or many

*Figure 5. GroupDb (Group Service)*

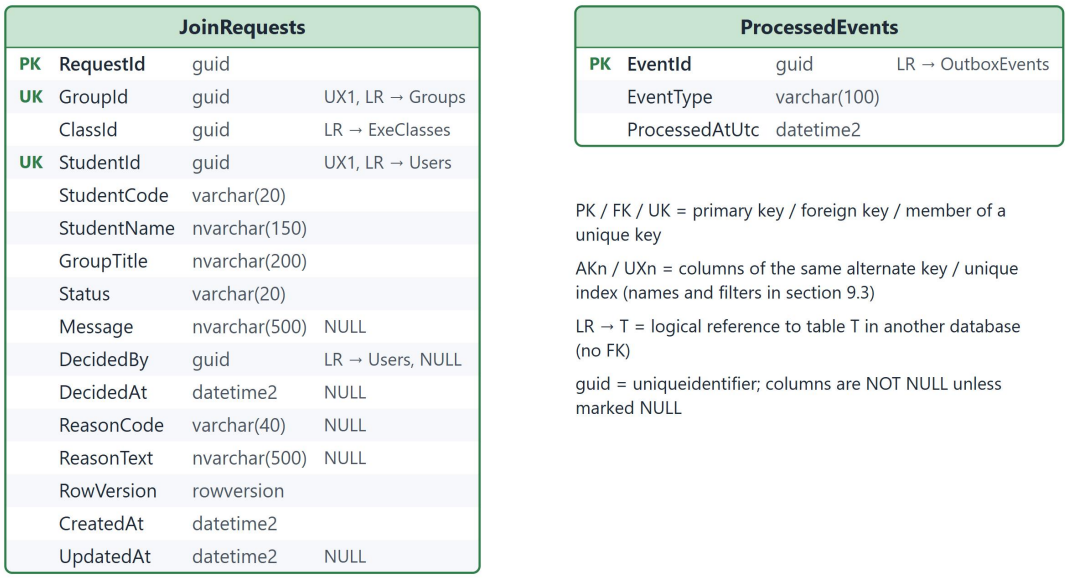



*Figure 6. RequestDb (Join Request Service)*

## 9.2. Data Dictionary

Abbreviations: PK primary key, FK foreign key, AK alternate key, UX unique index, IX non-unique index, CK check constraint, LR logical reference (section 9.4). All timestamps are UTC.

### 9.2.1. IdentityDb (owned by Identity Service)

*Table 22. IdentityDb.Users – User accounts for all roles*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **UserId** | uniqueidentifier | No | PK | Surrogate key (sequential GUID generated by EF Core). |
| **Email** | nvarchar(255) | No | UX_ Users_ Email | University e-mail used to sign in. |
| **UserCode** | varchar(20) | Yes | UX_ Users_ UserCode (filtered) | Student code (e.g. SE180001) or staff code; required for Student accounts (validated by Identity Service); unique when present. |
| **PasswordHash** | nvarchar(500) | No | – | Salted PBKDF2 hash (ASP.NET Core PasswordHasher). |
| **FullName** | nvarchar(150) | No | – | Display name. |
| **Status** | varchar(20) | No | Default 'Active'; CK_ Users_ Status | Account state: Active, Locked. |
| **CreatedAt** | datetime2 | No | Default SYSUTCDATETIME() | Creation time (UTC). |
| **UpdatedAt** | datetime2 | Yes | – | Last update time (UTC). |

*Table 23. IdentityDb.Roles – Global roles (seeded): Student, Lecturer, Administrator*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **RoleId** | uniqueidentifier | No | PK | Surrogate key (fixed seed values). |
| **Name** | varchar(50) | No | UX_ Roles_ Name | Role name: Student, Lecturer, Administrator. |

*Table 24. IdentityDb.UserRoles – Many-to-many assignment of roles to users*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **UserId** | uniqueidentifier | No | PK; FK → Users(UserId) | References Users(UserId); cascade delete. |
| **RoleId** | uniqueidentifier | No | PK; FK → Roles(RoleId) | References Roles(RoleId). |

*Table 25. IdentityDb.RefreshTokens – Rotating refresh tokens; only a hash is stored*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **RefreshTokenId** | uniqueidentifier | No | PK | Surrogate key. |
| **UserId** | uniqueidentifier | No | FK → Users(UserId); IX_RefreshTokens_UserId | References Users(UserId); cascade delete. |
| **TokenHash** | char(64) | No | UX_RefreshTokens_TokenHash | SHA-256 (hex) of the refresh token; the token itself is never stored. |
| **ExpiresAt** | datetime2 | No | – | Expiry (UTC); default lifetime 7 days (Jwt:RefreshTokenDays). |
| **CreatedAt** | datetime2 | No | Default SYSUTCDATETIME() | Issue time (UTC). |
| **RevokedAt** | datetime2 | Yes | – | Set when the token is rotated, on sign-out or when the account is locked. |
| **ReplacedByTokenId** | uniqueidentifier | Yes | – | Token issued when this one was rotated; reusing a revoked token revokes the whole chain. |

### 9.2.2. GroupDb (owned by Group Service)

*Table 26. GroupDb.Semesters – Academic semesters*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **SemesterId** | uniqueidentifier | No | PK | Surrogate key. |
| **Code** | varchar(20) | No | UX_ Semesters_ Code | Short code, e.g. FA26. |
| **Name** | nvarchar(100) | No | – | Full name, e.g. Fall 2026. |
| **StartDate** | date | No | CK_ Semesters_ Dates | First day of the semester. |
| **EndDate** | date | No | CK_ Semesters_ Dates | Last day of the semester; must be after StartDate. |

*Table 27. GroupDb.Courses – EXE courses*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **CourseId** | uniqueidentifier | No | PK | Surrogate key. |
| **Code** | varchar(20) | No | UX_ Courses_ Code | Course code, e.g. EXE101, EXE201. |
| **Name** | nvarchar(150) | No | – | Course title. |

*Table 28. GroupDb.ExeClasses – Class sections of an EXE course in a semester, with grouping rules*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **ClassId** | uniqueidentifier | No | PK | Surrogate key. |
| **SemesterId** | uniqueidentifier | No | FK → Semesters(SemesterId); UX_ ExeClasses_ Semester_ Course_ Code | References Semesters(SemesterId). |
| **CourseId** | uniqueidentifier | No | FK → Courses(CourseId); UX_ ExeClasses_ Semester_ Course_ Code | References Courses(CourseId). |
| **ClassCode** | varchar(30) | No | UX_ ExeClasses_ Semester_ Course_ Code | Class code, e.g. SE1801; unique per semester and course. |

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **LecturerId** | uniqueidentifier | No | LR → Users | Assigned lecturer → IdentityDb.Users (logical reference). |
| **MinMembers** | int | No | CK_ ExeClasses_ Members | Groups below this size are reported as under-filled. |
| **MaxMembers** | int | No | CK_ ExeClasses_ Members | Maximum group size in this class (≥ MinMembers). |
| **Deadline** | datetime2 | No | – | Grouping deadline (UTC); can change only while the class is Open. |
| **Status** | varchar(20) | No | Default 'Open'; CK_ ExeClasses_ Status | Open, Closed (set by the deadline job). |
| **CreatedAt** | datetime2 | No | Default SYSUTCDATETIME() | Creation time (UTC). |

*Table 29. GroupDb.ClassEnrollments – Class roster: which students belong to which class*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **ClassId** | uniqueidentifier | No | PK; FK → ExeClasses(ClassId) | References ExeClasses(ClassId). |
| **StudentId** | uniqueidentifier | No | PK; LR → Users | Student → IdentityDb.Users (logical reference). |
| **StudentCode** | varchar(20) | No | – | Snapshot of Users.UserCode; refreshed when the roster is imported again. |
| **FullName** | nvarchar(150) | No | – | Snapshot of Users.FullName, used for rosters, member lists and join requests. |
| **Email** | nvarchar(255) | No | – | Snapshot of Users.Email. |
| **EnrolledAt** | datetime2 | No | Default SYSUTCDATETIME() | Enrollment time (UTC). |

*Table 30. GroupDb.Groups – Project groups formed inside a class*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **GroupId** | uniqueidentifier | No | PK; AK_ Groups_ GroupId_ ClassId | Surrogate key. |
| **ClassId** | uniqueidentifier | No | FK → ExeClasses(ClassId); AK_ Groups_ GroupId_ ClassId; UX_ Groups_ Class_ Title (filtered) | References ExeClasses(ClassId). |
| **LeaderId** | uniqueidentifier | No | LR → Users | Team Leader → IdentityDb.Users; a member while the group is not Dissolved (kept for history afterwards). |
| **Title** | nvarchar(200) | No | UX_ Groups_ Class_ Title (filtered) | Project topic / group name; unique in a class among groups that are not Dissolved. |
| **Description** | nvarchar(1000) | Yes | – | Startup idea and wanted skills. |
| **CurrentMemberCount** | int | No | Default 1; CK_ Groups_ MemberCount | Cached head-count used for the capacity check; 0 only when Dissolved. |
| **Status** | varchar(20) | No | Default 'Recruiting'; CK_ Groups_ Status; CK_ Groups_ MemberCount | Recruiting, Full, Locked, Dissolved. |
| **RowVersion** | rowversion | No | Concurrency token | Optimistic concurrency token. |

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **CreatedAt** | datetime2 | No | Default SYSUTCDATETIME() | Creation time (UTC). |
| **UpdatedAt** | datetime2 | Yes | – | Last update time (UTC). |

*Table 31. GroupDb.GroupMembers – Membership of enrolled students in groups*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **MembershipId** | uniqueidentifier | No | PK | Surrogate key. |
| **GroupId** | uniqueidentifier | No | FK → Groups(GroupId, ClassId) | With ClassId references Groups(GroupId, ClassId). |
| **ClassId** | uniqueidentifier | No | FK → Groups(GroupId, ClassId); FK → ClassEnrollments(ClassId, StudentId); UX_GroupMembers_Class_Student | Must equal the group's class (composite FK); part of the one-group-per-class key. |
| **StudentId** | uniqueidentifier | No | FK → ClassEnrollments(ClassId, StudentId); UX_GroupMembers_Class_Student; LR → Users | With ClassId references ClassEnrollments (only enrolled students); student → IdentityDb.Users. |
| **JoinedAt** | datetime2 | No | Default SYSUTCDATETIME() | Time the membership was created (UTC). |

*Table 32. GroupDb.AddMemberRequests – Idempotency records of successful AddMemberIfEligible calls (one per join request)*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **RequestId** | uniqueidentifier | No | PK; LR → JoinRequests | = RequestDb.JoinRequests.RequestId (request_id of AddMemberIfEligible) – the idempotency key. |
| **GroupId** | uniqueidentifier | No | FK → Groups(GroupId) | References Groups(GroupId); the group the student was added to. |
| **StudentId** | uniqueidentifier | No | LR → Users | Student who was added → IdentityDb.Users. |
| **MembershipId** | uniqueidentifier | No | – | GroupMembers row created by this request (no FK: the membership may end later). |
| **ProcessedAtUtc** | datetime2 | No | Default SYSUTCDATETIME() | Time the member was added; rows are cleaned up after 30 days. |

*Table 33. GroupDb.OutboxEvents – Transactional outbox: events written in the same transaction as the business change*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **EventId** | uniqueidentifier | No | PK | Event id; idempotency key on the consumer side. |
| **EventType** | varchar(100) | No | – | GroupMemberAdded or GroupClosed. |
| **AggregateId** | uniqueidentifier | No | – | GroupId the event is about (no FK). |
| **Payload** | nvarchar(max) | No | CK_OutboxEvents_Payload | JSON payload (CHECK ISJSON). |
| **CorrelationId** | varchar(64) | Yes | – | Correlation id of the originating request. |
| **OccurredAtUtc** | datetime2 | No | Default SYSUTCDATETIME() | Time the event was recorded. |

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **PublishedAtUtc** | datetime2 | Yes | – | NULL until published to the Redis stream. |
| **RetryCount** | int | No | Default 0 | Failed publish attempts. |
| **LastError** | nvarchar(1000) | Yes | – | Last publish error, for diagnostics. |

### 9.2.3. RequestDb (owned by Join Request Service)

*Table 34. RequestDb.JoinRequests – Requests from students to join a group, with their lifecycle state*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **RequestId** | uniqueidentifier | No | PK | Surrogate key; also the idempotency key of AddMemberIfEligible. |
| **GroupId** | uniqueidentifier | No | UX_ JoinRequests_ ActiveRequest (filtered); LR → Groups | Target group → GroupDb.Groups (logical reference). |
| **ClassId** | uniqueidentifier | No | LR → ExeClasses | Class of the group, copied from GetGroupContext at submit time. |
| **StudentId** | uniqueidentifier | No | UX_ JoinRequests_ ActiveRequest (filtered); LR → Users | Requesting student (JWT sub) → IdentityDb.Users. |
| **StudentCode** | varchar(20) | No | – | Snapshot of the requester's student code (from GetGroupContext). |
| **StudentName** | nvarchar(150) | No | – | Snapshot of the requester's full name, shown to the Team Leader. |
| **GroupTitle** | nvarchar(200) | No | – | Snapshot of the group title, shown in the student's request list. |
| **Status** | varchar(20) | No | Default 'Pending'; CK_ JoinRequests_ Status | Pending, Processing, Approved, Rejected, Cancelled. |
| **Message** | nvarchar(500) | Yes | – | Optional introduction from the student. |
| **DecidedBy** | uniqueidentifier | Yes | LR → Users | Team Leader who approved/rejected → IdentityDb.Users. |
| **DecidedAt** | datetime2 | Yes | – | Time of the approve/reject decision; for Processing, when the request entered Processing (used by ProcessingReconciler). |
| **ReasonCode** | varchar(40) | Yes | CK_ JoinRequests_ ReasonCode | Machine-readable reason (list in section 5.3). |
| **ReasonText** | nvarchar(500) | Yes | – | Free text from the Team Leader when rejecting. |
| **RowVersion** | rowversion | No | Concurrency token | Optimistic concurrency token for state transitions. |
| **CreatedAt** | datetime2 | No | Default SYSUTCDATETIME() | Submission time (UTC). |
| **UpdatedAt** | datetime2 | Yes | – | Last state change (UTC). |

*Table 35. RequestDb.ProcessedEvents – Inbox of consumed event ids (idempotent consumer)*

| Column | Data type | Null | Key / constraint | Description |
|---|---|---|---|---|
| **EventId** | uniqueidentifier | No | PK; LR → OutboxEvents | Id of a consumed event = GroupDb.OutboxEvents.EventId (carried in the stream message). |
| **EventType** | varchar(100) | No | – | GroupMemberAdded or GroupClosed. |
| **ProcessedAtUtc** | datetime2 | No | Default SYSUTCDATETIME() | Time the event was applied; rows are cleaned up after 30 days. |

## 9.3. Indexes, Constraints & Business-Rule Matrix

*Table 36. Indexes, constraints and business rules*

| Rule | Implementation (SQL Server / EF Core) | Behavior on violation or conflict |
|---|---|---|
| **BR-01 Enrollment** | `FK_ GroupMembers_ ClassEnrollments` (ClassId, StudentId) → ClassEnrollments; `GetGroupContext. student_ enrolled` on submit | Insert rejected → NOT_ ENROLLED (409) |
| **BR-02 One group per class** | Membership lookup first (idempotency); `UX_ GroupMembers_ Class_ Student` (ClassId, StudentId) as the last line of defense | Same group → ALREADY_ ADDED; other group → ALREADY_ IN_ OTHER_ GRO UP (409); a duplicate-key error 2601 from a race is re-read and mapped the same way |
| **Member class = group class** | `AK_ Groups_ GroupId_ ClassId` + `FK_ GroupMembers_ Groups` (GroupId, ClassId) | Insert rejected (programming error) → 500, logged |
| **BR-03 Leadership** | `Groups. LeaderId` NOT NULL; transfer, leave and dissolve check “leader is a member” in one transaction; `CK_ Groups_ MemberCount` (count 0 only when Dissolved) | 409; dissolving deletes the last membership in the same transaction |
| **BR-04 Capacity** | CurrentMemberCount < MaxMembers checked in the transaction; `Groups. RowVersion` concurrency token (`IsRowVersion()`) | Concurrency conflict → reload and retry up to 3 times; GROUP_ FULL when no slot is left; ABORTED (transient) after 3 conflicts |
| **BR-05 Deadline** | Deadline (UTC) compared in every membership transaction; DeadlineEnforcer sets ExeClasses.Status = Closed and Groups.Status = Locked | DEADLINE_ PASSED or GROUP_ LOCKED (409) |
| **BR-06 One active request** | `UX_ JoinRequests_ ActiveRequest` (GroupId, StudentId) WHERE Status IN ('Pending', 'Processing') | Duplicate-key error 2601 → 409, request already active |
| **BR-07 Automatic cleanup** | Consumer updates through `IX_ JoinRequests_ Student_ Class_ Status` (T8) and `IX_ JoinRequests_ Group_ Status` (T9), only for requests with CreatedAt < occurredAtUtc | – |
| **BR-08 / BR-09 Class settings** | ExeClasses.MinMembers, MaxMembers, Deadline + `CK_ ExeClasses_ Members`; PUT /api/v1/classes/{id} recalculates group statuses in one transaction | 409 when the new MaxMembers is below the largest group or the class is Closed |
| **Request transitions** | `UPDATE … WHERE RequestId = @id AND Status = @expected` + `JoinRequests. RowVersion` | 0 rows affected → 409 Conflict |
| **Idempotent AddMemberIfEligible** | `AddMemberRequests` (PK RequestId) written with GroupMembers and OutboxEvents in one transaction | Repeated request_ id → ALREADY_ ADDED, no second event |
| **Unique names and codes** | `UX_ Groups_ Class_ Title` (WHERE Status <> 'Dissolved'), `UX_ ExeClasses_ Semester_ Course_ Code`, `UX_ Semesters_ Code`, `UX_ Courses_ Code`, `UX_ Roles_ Name`, `UX_ Users_ Email`, `UX_ Users_ UserCode` (WHERE UserCode IS NOT NULL), UX_RefreshTokens_TokenHash | Duplicate-key error 2601 → 409 with the conflicting field |

| Rule | Implementation (SQL Server / EF Core) | Behavior on violation or conflict |
|---|---|---|
| **Valid values** | CHECK constraints: status columns, reason codes, EndDate > StartDate, 1 ≤ MinMembers ≤ MaxMembers, member count vs Dissolved, ISJSON(Payload) | Validated in the API first (400); the CHECK is the last line of defense |
| **Transactional outbox** | GroupMembers, Groups, AddMemberRequests and OutboxEvents written in one SaveChanges transaction; `IX_ OutboxEvents_ Unpublished` (OccurredAtUtc) WHERE PublishedAtUtc IS NULL | Broker down → events stay unpublished and are retried (RetryCount, LastError) |
| **Idempotent consumer** | INSERT ProcessedEvents (PK EventId) and UPDATE JoinRequests in one transaction; XACK only after commit | Primary-key error 2627 → duplicate, skipped and acknowledged |
| **No cascading deletes** | All foreign keys ON DELETE NO ACTION (EF `DeleteBehavior. NoAction`), except UserRoles → Users and RefreshTokens → Users; DELETE /api/v1/classes/{id} only without groups, deleting the roster in the same transaction | Avoids SQL Server's “multiple cascade paths” error; blocked deletes return 409 |
| **Supporting indexes** | `IX_ UserRoles_ RoleId` (RoleId)<br>IX_RefreshTokens_UserId (UserId)<br>`IX_ ExeClasses_ CourseId` (CourseId)<br>`IX_ ExeClasses_ LecturerId` (LecturerId)<br>`IX_ ClassEnrollments_ StudentId` (StudentId)<br>`IX_ Groups_ Class_ Status` (ClassId, Status)<br>`IX_ Groups_ LeaderId` (LeaderId)<br>`IX_ GroupMembers_ Group_ Class` (GroupId, ClassId)<br>`IX_ AddMemberRequests_ GroupId` (GroupId)<br>`IX_ AddMemberRequests_ ProcessedAtUtc` (ProcessedAtUtc)<br>`IX_ OutboxEvents_ Unpublished` (OccurredAtUtc) WHERE PublishedAtUtc IS NULL<br>`IX_ OutboxEvents_ PublishedAtUtc` (PublishedAtUtc) WHERE PublishedAtUtc IS NOT NULL<br>`IX_ JoinRequests_ Student_ Class_ Status` (StudentId, ClassId, Status)<br>`IX_ JoinRequests_ Group_ Status` (GroupId, Status)<br>`IX_ JoinRequests_ Processing` (DecidedAt) WHERE Status = 'Processing'<br>`IX_ ProcessedEvents_ ProcessedAtUtc` (ProcessedAtUtc) | Paging and filtering, dashboard queries, jobs and cleanup |

## 9.4. Cross-Service Logical References

*Table 37. Logical references between databases*

| Reference | Target | How integrity is kept |
|---|---|---|
| `GroupDb. ExeClasses. LecturerId` | IdentityDb.Users | The Administrator picks a user with the Lecturer role when creating or updating the class. |
| `GroupDb. ClassEnrollments. StudentId` | IdentityDb.Users | The Administrator enrolls existing Student accounts chosen from GET /api/v1/users?role=Student; code, name and e-mail are stored as snapshots and refreshed when the roster is imported again. |
| `GroupDb. Groups. LeaderId` | IdentityDb.Users | A member of the group while it is not Dissolved (checked in Group Service). |
| `GroupDb. GroupMembers. StudentId` | IdentityDb.Users | The FK to ClassEnrollments guarantees the student is enrolled in the class. |
| `GroupDb. AddMemberRequests. StudentId` | IdentityDb.Users | Copied from the AddMemberIfEligible request. |
| `GroupDb. AddMemberRequests. RequestId` | RequestDb.JoinRequests.RequestId | The request_ id of AddMemberIfEligible, sent only by Join Request Service (approve, ProcessingReconciler); used as the idempotency key, never looked up. |

| Reference | Target | How integrity is kept |
|---|---|---|
| `RequestDb. JoinRequests. GroupId / ClassId` | GroupDb.Groups / ExeClasses | Validated through gRPC GetGroupContext when the request is submitted; StudentCode, StudentName and GroupTitle are snapshots from the same call. |
| `RequestDb. JoinRequests. StudentId / DecidedBy` | IdentityDb.Users | Taken from the validated JWT (sub claim) of the caller. |
| `RequestDb. ProcessedEvents. EventId` | GroupDb.OutboxEvents. EventId | Carried in the Redis Streams message envelope; used for de-duplication. |

### 9.5. Migrations & Reference DDL

- Each service keeps its own DbContext and migrations (`dotnet ef migrations add InitialCreate`), applied at start-up in Development.
- EF Core mapping of the key constraints: `HasAlternateKey(g => new { g.GroupId, g.ClassId })` with `HasForeignKey(...).HasPrincipalKey(...)` for the composite FKs, `IsRowVersion()`, `HasIndex(...).IsUnique().HasFilter(...)`, `ToTable(t => t.HasCheckConstraint(...))` and `OnDelete(DeleteBehavior.NoAction)`.
- The reference DDL (`database/schema.sql`) mirrors the same model. It was executed on SQL Server 2019 (LocalDB) and passed 34 tests (constraints, concurrency and key queries) – for example a second group in the same class, a non-enrolled member, a member whose class differs from the group's class, a second active request, a repeated request_id, invalid status or reason codes, and an update with a stale RowVersion were all rejected, while the dissolve flow and the ungrouped-students query behaved as designed.

## Appendix A. gRPC Contract – group_membership.proto

```
syntax = "proto3";

package exegroup.v1;

import "google/protobuf/timestamp.proto";

option csharp_namespace = "ExeGroup.Contracts.Grpc";

// Served by Group Service on http://group-service:8081 (HTTP/2 only, internal network);
// called by Join Request Service. All ids are GUID strings.
service GroupMembership {
  // Read-only snapshot: eligibility and display data on submit,
  // leadership check on list, approve and reject.
  rpc GetGroupContext (GetGroupContextRequest) returns (GetGroupContextReply);

  // Idempotent by request_id. Order of checks:
  // (1) request_id already processed, or student already in this group -> ALREADY_ADDED;
  // (2) student already in another group of the class -> ALREADY_IN_OTHER_GROUP;
  // (3) group, leader, enrollment, deadline, status and capacity rules;
  // (4) insert GroupMembers, AddMemberRequests and OutboxEvents in one transaction.
  rpc AddMemberIfEligible (AddMemberIfEligibleRequest) returns (AddMemberIfEligibleReply);
}

enum GroupStatus {
  GROUP_STATUS_UNSPECIFIED = 0;
  GROUP_STATUS_RECRUITING = 1;
  GROUP_STATUS_FULL = 2;
  GROUP_STATUS_LOCKED = 3;
  GROUP_STATUS_DISSOLVED = 4;
}

message GetGroupContextRequest {
```

```
  string group_id = 1;
  string student_id = 2;  // optional: empty when only the leader is checked
}

message GetGroupContextReply {
  bool exists = 1;
  string class_id = 2;
  string leader_id = 3;
  GroupStatus group_status = 4;
  google.protobuf.Timestamp deadline_utc = 5;
  int32 current_member_count = 6;
  int32 max_members = 7;
  string group_title = 8;
  // Fields 9-12 describe student_id; they are empty when student_id is empty.
  bool student_enrolled = 9;
  bool student_has_group_in_class = 10;
  string student_code = 11;       // snapshot from ClassEnrollments
  string student_full_name = 12;  // snapshot from ClassEnrollments
}

message AddMemberIfEligibleRequest {
  string request_id = 1;   // join request id = idempotency key
  string group_id = 2;
  string student_id = 3;
  string approver_id = 4;  // sub of the Team Leader, validated by Join Request Service
}

enum AddMemberResult {
  ADD_MEMBER_RESULT_UNSPECIFIED = 0;
  ADD_MEMBER_RESULT_ADDED = 1;
  ADD_MEMBER_RESULT_ALREADY_ADDED = 2;
  ADD_MEMBER_RESULT_GROUP_FULL = 3;
  ADD_MEMBER_RESULT_GROUP_LOCKED = 4;
  ADD_MEMBER_RESULT_GROUP_DISSOLVED = 5;
  ADD_MEMBER_RESULT_DEADLINE_PASSED = 6;
  ADD_MEMBER_RESULT_ALREADY_IN_OTHER_GROUP = 7;
  ADD_MEMBER_RESULT_NOT_ENROLLED = 8;
  ADD_MEMBER_RESULT_NOT_LEADER = 9;
  ADD_MEMBER_RESULT_GROUP_NOT_FOUND = 10;
}

message AddMemberIfEligibleReply {
  AddMemberResult result = 1;
  string membership_id = 2;  // set for ADDED and ALREADY_ADDED
  int32 current_member_count = 3;
  google.protobuf.Timestamp added_at_utc = 4;  // set for ADDED and ALREADY_ADDED: when the member
was added
}
```

# Appendix B. Glossary

*Table 38. Glossary*

| Term | Meaning |
|---|---|
| **C4 model** | Notation for software architecture diagrams at four zoom levels; this document uses Level 1 (System Context) and Level 2 (Container). |
| **Container (C4)** | A separately deployable or runnable unit – an application, a database or a message stream. |
| **CRUD** | Create, read, update, delete. |
| **DDL** | Data Definition Language – the SQL script that creates tables, keys and indexes. |
| **EXE** | Experiential Entrepreneurship courses (EXE101, EXE201) in which students build a startup project in groups. |
| **gRPC / h2c** | High-performance RPC over HTTP/2 with Protocol Buffers; h2c = HTTP/2 without TLS, used inside Docker. |
| **JWT** | JSON Web Token – signed access token carrying the user id (sub) and roles. |

| Term | Meaning |
|---|---|
| **Key prefixes** | PK primary key, FK foreign key, AK alternate key, UX unique index, IX non-unique index, CK check constraint. |
| **LMS** | Learning Management System of the university (out of scope). |
| **LR (logical reference)** | A column that refers to a row in another service's database without a foreign key. |
| **MoSCoW** | Prioritization scheme used in section 5.1: Must have, Should have (Could / Won't are not used). |
| **MVP** | Minimum Viable Product – the scope delivered in this assignment. |
| **Outbox / idempotent consumer** | Patterns that publish events reliably (written in the same transaction as the data) and apply each event only once (ProcessedEvents). |
| **Redis Streams** | Persistent append-only log in Redis with consumer groups and acknowledgements. |
| **SPA** | Single-Page Application – here the React “Single-Page App” container. |
| **SSO** | Single sign-on with the university identity provider (out of scope). |
| **UTC** | Coordinated Universal Time; all timestamps and deadlines are stored in UTC. |
| **YARP** | Yet Another Reverse Proxy – Microsoft's .NET library used to build the API Gateway. |

# Appendix C. Generated architecture views

Generated from docs/architecture/workspace.dsl with Structurizr and C4-PlantUML (regenerate with the commands in the README). Components marked [Planned] are designed but not implemented yet.

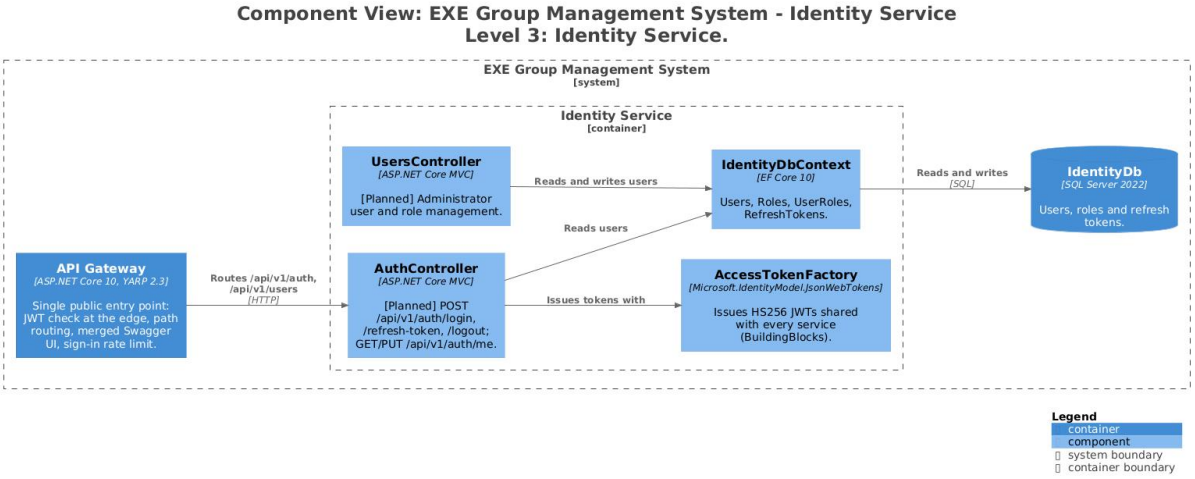



*Figure 7. Level 3 – Identity Service components*

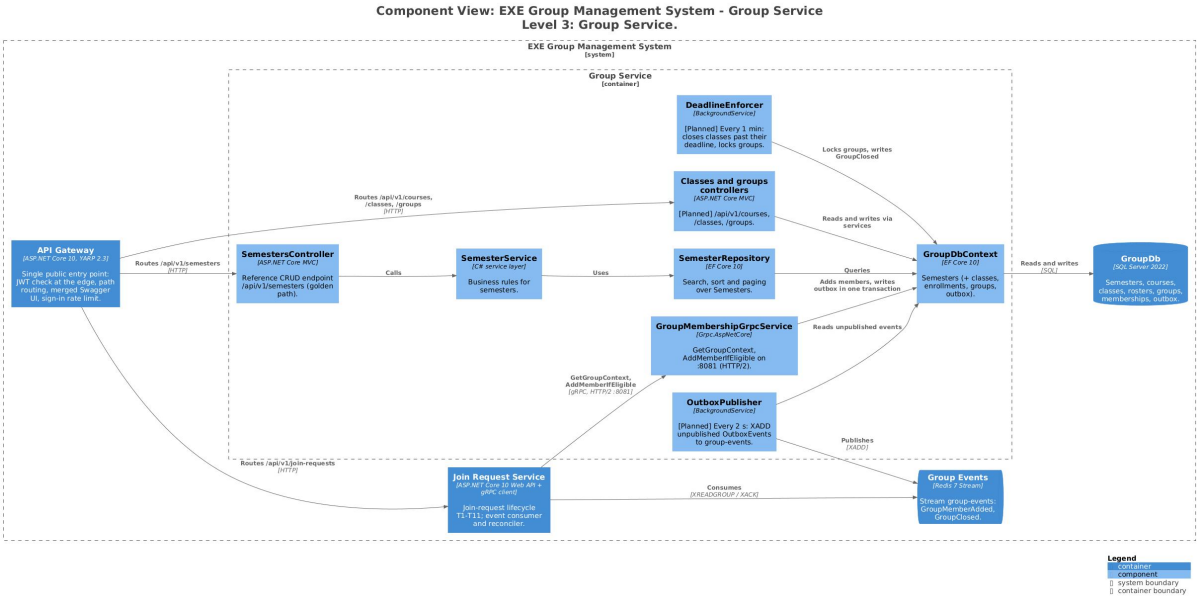



*Figure 8. Level 3 – Group Service components*

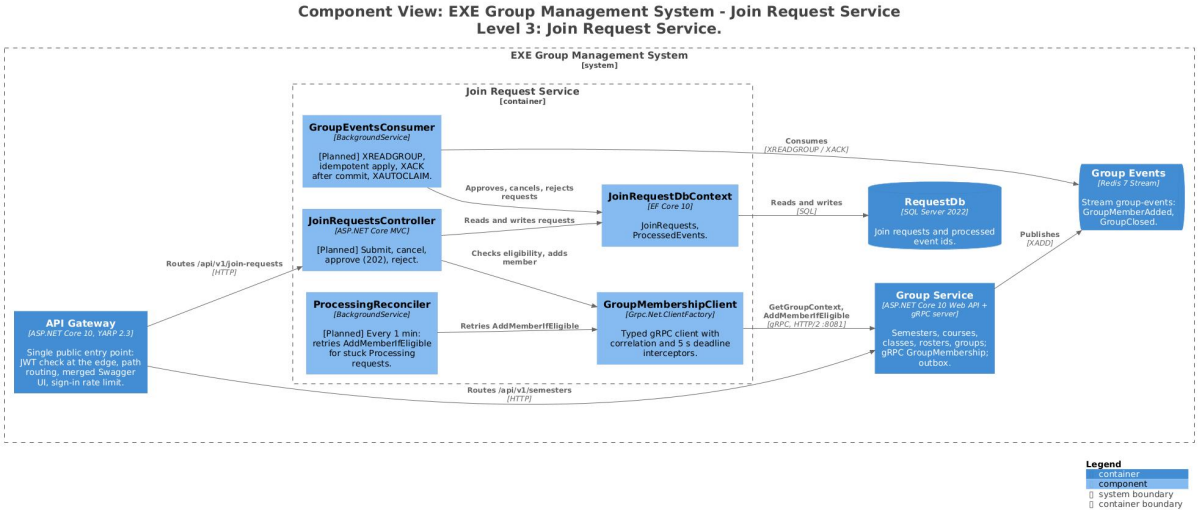



*Figure 9. Level 3 – Join Request Service components*

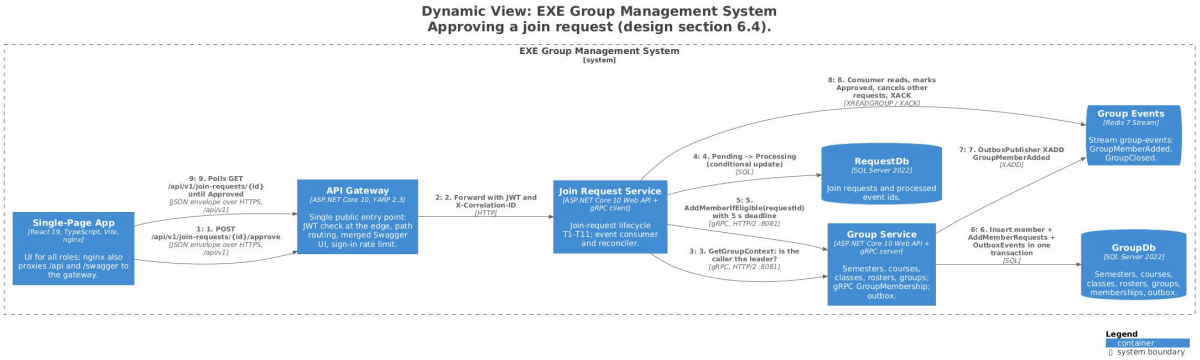



*Figure 10. Dynamic view – approving a join request*

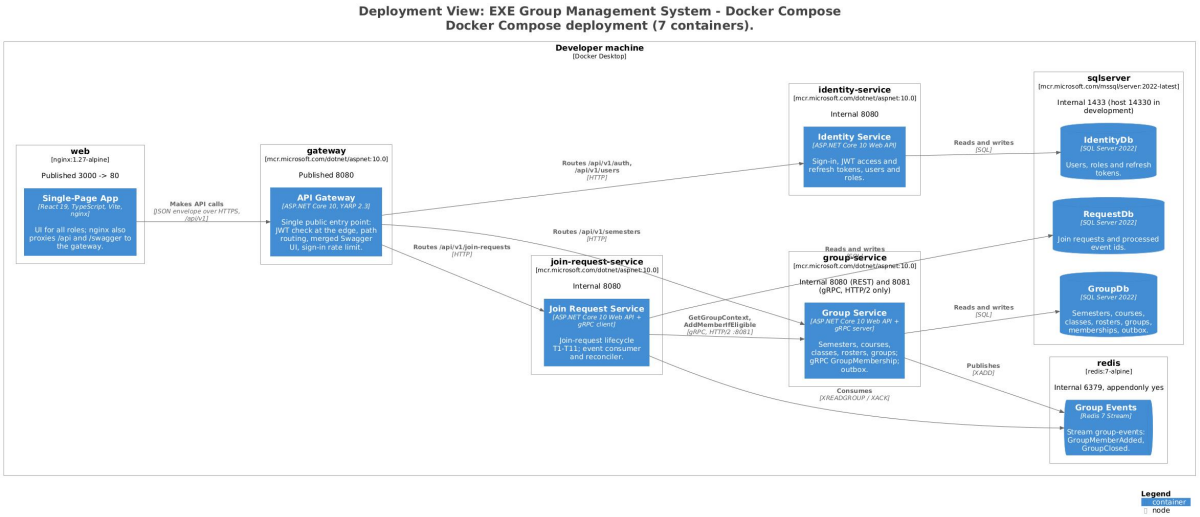



*Figure 11. Deployment view – Docker Compose*